SIMATS ENGINEERING

ARTIFICIAL INTELLIGENCE (CSA17)

CO2 – Scenario Based Assignment: Answers

Student Name: Abinekavarshini.M

Register Number: 1924260333

Note: These answers follow the five scenarios and tasks in the uploaded assignment.

1. AI-Powered Drones for Emergency Medicine Delivery

i. Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search selects the next location with the smallest heuristic value, h(n). In this scenario, h(n) is the straight-line distance from the drone's current location to the medicine destination. It tries to move toward the destination that appears closest.

Strengths: It is often fast, simple, and useful when a quick route estimate is needed. It may reach a goal quickly when the heuristic is informative.

Weaknesses: It considers the heuristic but does not properly account for the total travel cost already incurred. The apparently shortest route may have a blocked road, dangerous weather, or difficult terrain. It is not guaranteed to find the lowest-cost or safest route.

ii. How A* balances cost and heuristic

A* evaluates each candidate location using f(n) = g(n) + h(n), where g(n) is the actual cost from the starting point to n and h(n) is the estimated remaining cost to the destination. It balances the cost already paid with the estimated cost still to come. The drone can assign higher movement costs to hazardous weather or difficult terrain and update its route when new information arrives.

iii. Impact of heuristic accuracy

A useful, accurate heuristic guides both algorithms toward promising routes. If the straight-line estimate poorly reflects rivers, blocked roads, wind, or terrain, Greedy Best-First Search may choose a misleading direction. A* can still be misled by a poor heuristic, but it also considers the accumulated path cost. With a suitable admissible heuristic and appropriate search conditions, A* can find an optimal path.

iv. Effect of dynamic blocked paths

A blocked path may make a planned route unusable. The drone must incorporate the new information, remove or increase the cost of affected edges, and search again or use an incremental replanning method. Repeated changes can increase computation and delay, so the system should prioritize fresh hazard data and safe alternative routes.

v. Recommendation

A* is the better general choice when the drone needs a balance of route cost and estimated remaining distance. For strict real-time operation, use A* with updated edge costs, a reliable heuristic, safety constraints, and fast replanning. Greedy search may be quicker in some cases, but its lack of cost awareness makes it less suitable where medicine delivery safety and route quality matter.

2. Smart-City Traffic Signal Optimization

Formulation as an optimization problem

The system can represent signal timings at intersections as decision variables. The objective is to minimize a combined cost such as J = w1(Total waiting time) + w2(Fuel consumption) + w3(Traffic congestion), where w1, w2, and w3 are weights that express the importance of each goal. Constraints include minimum and maximum green

times, pedestrian crossing needs, signal safety rules, and coordination between intersections.

i. Hill Climbing and its limitations

Hill Climbing starts with a set of signal timings, evaluates nearby alternatives, and moves to a neighbour that improves the objective. It repeats until no neighbouring solution looks better. It is simple and can improve a solution quickly, but it may stop at a locally good solution instead of the best overall solution. Its result depends on the starting timings and neighbourhood design.

ii. Local maxima, plateaus and ridges

Local optimum: One intersection's timing may improve nearby traffic but worsen traffic across the wider network, so the algorithm stops at a solution that is not globally best. **Plateau:** Several timing changes produce almost the same score, giving the algorithm no clear direction. **Ridge:** Improvement may require coordinated changes across several intersections, while changing only one signal at a time appears unhelpful.

iii. Simulated Annealing or Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution early in the search, with a probability that decreases over time. This helps it escape local optima and gradually focus on better solutions. A Genetic Algorithm maintains a population of timing plans and uses selection, crossover, and mutation to create new candidates. Both can explore more broadly than basic Hill Climbing, although they require careful parameter settings and computation.

iv. Recommendation

A practical choice is a Genetic Algorithm for periodic network-wide optimization, combined with a fast local controller for immediate signal adjustments. The population-based search can explore many combinations and adapt when traffic patterns change. Simulated Annealing is a simpler alternative when computational resources are limited. The system should validate every candidate against traffic-safety and signal constraints before deployment.

3. Autonomous Mars Rover in Unknown Terrain

i. Why an online search agent is required

Offline search assumes that the environment and map are known before planning. The rover has only limited sensor information and discovers obstacles as it moves. An online search agent can act, observe the result, update its knowledge, and choose its next action. This suits an unknown environment where a complete map is unavailable.

ii. Partial observability and dynamic-environment challenges

Sensors may not detect every crater or rock, and readings may be incomplete or noisy. The rover must make decisions using uncertain information. Terrain, lighting, communication delays, and limited energy can also affect movement. A route that seemed safe may become unsuitable after new observations, so the rover must continually reassess risk.

iii. Building and updating an internal model

The rover maintains an internal map or state model containing known traversable areas, hazards, uncertain regions, sample locations, and estimated movement costs. After each sensing or movement action, it updates the model with new observations. It can mark confirmed obstacles as blocked, revise uncertain estimates, and replan toward the goal.

iv. Online search compared with uninformed and informed search

Uninformed search uses no goal-directed heuristic and may explore many states. Informed search uses a heuristic, such as estimated distance to a sample, to prioritize promising states. Online search describes the agent's ability to search while interacting with an incompletely known environment; it may use either uninformed or informed methods. An informed online strategy can reduce unnecessary exploration while adapting to new discoveries.

v. Recommendation

Use an informed online agent with a safe exploration policy, updated mapping, and frequent replanning. The heuristic should estimate distance or energy cost, while a separate risk measure penalizes hazards and uncertain terrain. The rover should choose conservative actions when sensor confidence is low and preserve enough energy to reach a safe state.

4. University Exam Timetabling as a Constraint Satisfaction Problem

Formulation: variables, domains and constraints

A Constraint Satisfaction Problem (CSP) consists of variables, domains of possible values, and constraints that restrict combinations of values.

Variables: Each exam is a variable. Its assigned value can be a time slot and an exam hall.

Domains: The possible time slots and halls available for each exam, after accounting for hall capacity, faculty availability, and special accommodations.

Constraints: Exams taken by the same student must not overlap; a hall cannot host two exams at the same time; hall capacity must be sufficient; prerequisite ordering must be respected; faculty must be available; and students requiring accommodations must be assigned suitable rooms or arrangements.

ii. Backtracking search

Backtracking assigns a time and hall to one exam at a time. After each assignment, it checks whether the constraints are still satisfied. If an assignment creates a conflict, the algorithm undoes it and tries another value. It continues until a complete valid timetable is found or all alternatives have been explored.

iii. Constraint propagation and forward checking

After assigning an exam, forward checking removes incompatible time slots or halls from the domains of related unassigned exams. If any variable has an empty domain, the current assignment cannot lead to a solution, so the algorithm backtracks immediately. This detects conflicts earlier and reduces wasted search.

iv. Real-world challenges and scalable strategy

Large universities may have many exams, students, rooms, and special cases. Constraints can conflict, and some requirements may be preferences rather than absolute rules. A scalable strategy is to use CSP backtracking with the minimum-remaining-values heuristic, degree heuristic, least-constraining-value ordering, and forward checking. Hard constraints should never be violated; soft preferences can be optimized after feasibility is achieved.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax models the problem

Minimax represents the game as a tree of possible moves. The AI is the maximizing player, trying to obtain the highest score, while the opponent is the minimizing player, trying to reduce that score. At terminal or depth-limited states, an evaluation function assigns a score. Scores are propagated upward: maximizing nodes choose the largest child value and minimizing nodes choose the smallest.

ii. Computational challenges of large game trees

If each state has b legal moves and the search depth is d, the number of nodes can grow roughly as O(b^d). Even a moderate branching factor becomes expensive at large depths. A strict time limit means the AI may be unable to search every possible continuation, and a depth-limited search depends on the quality of its evaluation function.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning produces the same Minimax decision as a full search at the same depth, but skips branches that cannot affect the final choice. Alpha is the best value found so far for the maximizing player; beta is the best value found so far for the minimizing player. When alpha is greater than or equal to beta, the remaining children of that node can be pruned because the opponent already has a choice at least as good elsewhere. Good move ordering increases pruning. In the best case, the effective search can approach O(b^(d/2)), allowing deeper search in the same time.

iv. Example evaluation function

A simple evaluation function for a strategy game is: Score(s) = 5 × (resource advantage) + 4 × (unit-strength advantage) + 3 × (territory advantage) + 2 × (objective progress) – 6 × (immediate threat to own base). The weights are illustrative and should be tuned using game testing. Resource and unit advantages support long-term strength, territory supports map control, objective progress measures progress toward winning, and threats to the base receive a strong penalty.

v. Balancing optimality and real-time performance

Use depth-limited Minimax with Alpha-Beta pruning, strong move ordering, and a time budget. Iterative deepening searches depth 1, then 2, and so on, keeping the best move from the last completed search if time runs out. A fast evaluation function and selective extensions for critical positions improve practical play. This approach does not guarantee a globally optimal move in an enormous game tree, but it provides a good balance of decision quality and speed.

Conclusion: The scenarios show that AI search methods should be selected according to cost, uncertainty, changing environments, constraints, and time limits. Heuristics guide search, replanning supports dynamic settings, constraint propagation reduces wasted work, and pruning makes adversarial search more efficient.